Лекция 12. Эволюционные алгоритмы и адаптивные системы

Отбор, мутация и обучение как способ проектировать поведение роботов

90 минут · две части по 45

Курс «Управление распределёнными робототехническими системами»

План лекции

1. Зачем эволюция инженеру и основные понятия
2. Генетический алгоритм: кодирование, отбор, скрещивание, мутация
3. Теорема о схемах Холланда
4. Эволюционные стратегии и CMA-ES
5. Дифференциальная эволюция и генетическое программирование
6. Многокритериальная оптимизация и NSGA-II
7. Сравнение методов и связь с роем частиц
8. Эволюционная робототехника и нейроэволюция
9. Целевая функция, назначение вклада и шум
10. Разрыв между симуляцией и реальностью
11. Онлайн- и воплощённая эволюция в роях роботов

Синие — первая часть · янтарные — вторая часть

Что нужно помнить с прошлых занятий

Сегодняшняя лекция опирается на шесть идей из прошлых занятий — освежим их до начала новой темы.

Понятие (лекция N)	Что это простыми словами	Где пригодится сегодня
Метод роя частиц (лекция 5)	Каждая частица тянется к своему лучшему и к лучшему у соседей решению	Сравним с генетическим отбором: рой никого не отбраковывает
Самоорганизация и эмерджентность (лекция 5)	Порядок в группе рождается из локальных правил, без центра	Эволюция подбирает такие локальные правила для роя
Исследование и использование (лекция 7)	Агент то пробует новое действие, то берёт лучшее известное (ε-жадно)	Тот же выбор делают мутация и скрещивание
Распределение заслуг (лекция 7)	При общей награде команды непонятно, чей вклад был полезен	Та же трудность возникает при эволюции поведения группы
Гомогенное и гетерогенное обучение (лекция 7)	Все роботы учат одну общую политику или каждый — свою	Выбор между одинаковыми и специализированными роботами
Топология связей и λ_2 (лекция 4)	Структура графа связи задаёт скорость согласования группы	Топология соседства в рое частиц задаёт характер поиска

Сокращения и термины лекции

Все сокращения, которые встретятся на слайдах сегодня, собраны здесь; к таблице можно вернуться в любой момент.

Обозначение	Расшифровка	Смысл в одной фразе
ЭА	Эволюционный алгоритм	Поиск решения отбором и случайными изменениями популяции
ГА	Генетический алгоритм (Холланд, начало 1970-х)	ЭА с хромосомами-строками, скрещиванием и мутацией
ЭС (ES)	Эволюционная стратегия, Evolution Strategy	ЭА для векторов вещественных чисел с гауссовой мутацией
CMA-ES	Covariance Matrix Adaptation ES — адаптация матрицы ковариации	ЭС, подстраивающая форму облака мутаций под рельеф функции
DE	Differential Evolution — дифференциальная эволюция	Шаг мутации строится из разности случайных векторов популяции
ГП	Генетическое программирование (Коза, начало 1990-х)	Эволюция программ и деревьев правил, а не чисел
NSGA-II	Non-dominated Sorting Genetic Algorithm II	ГА, строящий весь фронт Парето компромиссов за один запуск

Сокращения и термины лекции (продолжение)

Здесь остальные сокращения и обозначения, которые появятся в формулах и примерах.

Обозначение	Расшифровка	Смысл в одной фразе
NEAT	NeuroEvolution of Augmenting Topologies	Эволюция весов и структуры нейросети от простой к сложной
PSO	Particle Swarm Optimization — метод роя частиц	Решения движутся к лучшему своему и соседскому результату
PPTC	Распределённая робототехническая система	Группа роботов без единого центра управления
μ и λ	Число родителей и число потомков в ЭС	(μ + λ) и (μ, λ) — две схемы выбора следующего поколения
σ	Сила мутации — стандартное отклонение шума	Насколько далеко потомок в среднем отходит от родителя
p_c и p_m	Вероятности скрещивания и мутации в ГА	Как часто применяют каждый из двух операторов
Схема H, o(H), δ(H)	Шаблон хромосомы, его порядок и определяющая длина	Описывают группу похожих хромосом в теореме Холланда

Эволюция как метод оптимизации

Как отбор и случайные изменения решают задачи, к которым не подступиться через производную

“

Эволюция не ищет решение по направлению производной — она **выращивает** его отбором. Там, где нет производной и нет модели, это иногда единственный путь.

Ключевая мысль первой части

1 Когда классические методы не работают

Начнём с вопроса, зачем инженеру медленный случайный поиск, если есть быстрые градиентные методы.

– Целевая функция не задана формулой: её значение мы узнаём только из эксперимента или прогона симуляции.

– Производная не существует или её нельзя вычислить, поэтому градиентный метод не знает, куда шагать.

– У функции много локальных экстремумов, и метод спуска застревает в ближайшем из них.

– Часть переменных дискретна (порядок обхода стеллажей), часть непрерывна (скорость робота), и общий метод подобрать трудно.

Пример

Качество работы складского робота нельзя продифференцировать по параметрам его контроллера: чтобы узнать, сколько коробок он доставил за смену и сколько раз столкнулся, его нужно запустить и посмотреть.

1 Основные понятия эволюционных вычислений

Слова в эволюционных вычислениях взяты из биологии, но у каждого есть точный технический смысл.

– **Популяция** — это набор из μ особей, то есть сразу нескольких пробных решений, которые алгоритм улучшает одновременно.

– **Генотип** — закодированная запись решения, например строка битов; **фенотип** — само решение, например коэффициенты регулятора.

– **Функция приспособленности** (фитнес-функция) ставит каждому решению в соответствие одно число — оценку его качества.

– **Генетические операторы** — отбор, скрещивание (кроссовер) и мутация — порождают новые особи из имеющихся.

Пример

Особь — контроллер складского робота с параметрами «скорость, дистанция торможения, порог датчика». Приспособленность — число коробок, доставленных за 10 минут симуляции, минус штраф за каждое столкновение.

1 Цикл эволюционного алгоритма

Все разновидности эволюционных алгоритмов устроены по одному циклу и различаются только деталями отдельных шагов.



Отбор задаёт направление поиска, а скрещивание и мутация поставляют разнообразие; без отбора поиск становится случайным блужданием.

2 Кодирование решения

Первое решение при построении генетического алгоритма (ГА) — как записать решение задачи в виде хромосомы.

– **Двоичное** кодирование записывает решение строкой битов; это классика ГА для задач «да/нет», например какие стеллажи включить в маршрут.

– **Вещественное** кодирование хранит вектор чисел и естественно для параметров регулятора: скоростей и коэффициентов усиления.

– **Перестановочное** кодирование хранит порядок элементов, например порядок обхода точек комплектации заказа.

– **Древовидное** кодирование хранит программу в виде дерева и используется в генетическом программировании.

Ключевая идея

Кодирование должно быть локальным: близкие генотипы должны давать близкие решения и близкую приспособленность. Если записать маршрут строкой битов, скрещивание будет постоянно порождать недопустимые маршруты.

2 Двоичное кодирование вещественного параметра

Как строка битов превращается в число, например в скорость робота? Для этого служит формула декодирования.

$$x = a + d \cdot \frac{b - a}{2^L - 1}$$

- Здесь x — значение параметра на отрезке [a, b], L — число бит, d — целое число от 0 до $2^L - 1$, записанное этими битами.
- Смысл: отрезок [a, b] делится на $2^L - 1$ равных шагов, а строка битов говорит, сколько шагов отложить от a.
- Число бит задаёт точность: шаг равен $(b - a)/(2^L - 1)$, и каждый лишний бит уменьшает его примерно вдвое.
- Пример: скорость от 0 до 2 м/с, L = 8 бит. Шаг ≈ 0,008 м/с, а строка 10000000 (d = 128) даёт x ≈ 1,004 м/с.

Вместо прямого двоичного кода часто берут код Грея: в нём соседние числа отличаются ровно одним битом, и малое изменение решения не требует менять много бит.

2 Методы отбора родителей

Отбор — двигатель алгоритма: именно он создаёт давление в сторону лучших решений. Вот четыре основных способа.

Метод	Как работает	Свойство и когда применять
Рулетка (пропорциональный)	Вероятность выбора пропорциональна приспособленности особи	Чувствителен к масштабу: один «гигант» быстро вытесняет всех
Турнир размера k	Берём k случайных особей (обычно 2–3), родителем становится лучшая	Прост и устойчив; чем больше k, тем сильнее давление отбора
Ранговый	Вероятность зависит от места особи в рейтинге, а не от значения	Не зависит от масштаба значений приспособленности
Элитизм	Несколько лучших особей переходят в новое поколение без изменений	Лучшее решение не теряется; элита — единицы процентов

На практике чаще всего берут турнирный отбор с элитизмом: турнир не требует нормировки и устойчив к шуму в оценках, а элита сохраняет лучшее найденное решение.

2 Отбор методом рулетки

Разберём самый наглядный способ отбора и его слабое место на числах из примера пособия.

$$p_i = \frac{f_i}{\sum_{j=1}^{N} f_j}$$

- Здесь p_i — вероятность выбрать особь i родителем, f_i — её приспособленность, N — размер популяции; сумма идёт по всем особям.
- Смысл: колесо рулетки делится на секторы, площадь которых пропорциональна приспособленности, и колесо крутят N раз.
- Пример: при f = 169, 576, 64 и 361 сумма равна 1170; лучшая особь получает p ≈ 0,49, а худшая — всего p ≈ 0,055.
- Слабость: если одна особь во много раз лучше других, она быстро заполняет популяцию копиями, и разнообразие теряется.

2 Скрещивание и мутация

Отбор выбрал родителей; теперь из них нужно получить потомков. Для этого служат два оператора с разными ролями.

– **Скрещивание** (кроссовер) обменивает части родителей: 11111 и 00000 с разрывом после третьего гена дают 11100 и 00011.

– Скрещивание применяют часто, с вероятностью p_c от 0,6 до 0,95: оно использует уже накопленные удачные фрагменты решений.

– **Мутация** случайно меняет гены: в двоичном генетическом алгоритме каждый бит инвертируется с малой вероятностью p_m, часто 1/L.

– Мутация поддерживает разнообразие популяции и в принципе позволяет попасть в любую точку пространства поиска.

Ключевая идея

Скрещивание отвечает за использование найденного, мутация — за исследование нового. Это та же дилемма, что на седьмой лекции решала ε-жадная стратегия: много мутаций превращает поиск в случайный, мало — ведёт к застою.

2 Пример: одно поколение генетического алгоритма вручную

Проверим, как три оператора работают вместе, на задаче из пособия: максимизировать $f(x) = x^2$ для целых x от 0 до 31.

Этап	Популяция (x в двоичном коде)	Средняя / лучшая f
Начальная популяция	A = 01101 (13), B = 11000 (24), C = 01000 (8), D = 10011 (19)	292,5 / 576
Отбор рулеткой	В пары попали (B, D) и (B, A); слабая особь C выбыла	B получает около 2 копий
Одноточечное скрещивание	11001 (25), 10010 (18), 11101 (29), 01000 (8)	463,5 / 841
Мутация старшего бита	01000 превращается в 11000 (24)	591,5 / 841

За одно поколение средняя приспособленность выросла с 292,5 до 591,5: отбор собрал материал вокруг сильных особей, а скрещивание соединило удачные старшие единичные биты.

3 Схема: шаблон для группы хромосом

Почему генетический алгоритм работает лучше слепого перебора? Ответ Холланда опирается на понятие схемы.

– **Схема H** — шаблон из символов 0, 1 и *, где звёздочка означает «любое значение»; схема описывает все подходящие под шаблон хромосомы.

– Например, схема 1**0 описывает все четырёхбитные строки, которые начинаются на 1 и кончаются на 0: 1000, 1010, 1100 и 1110.

– **Порядок схемы o(H)** — число фиксированных позиций, то есть не звёздочек: у схемы 1**0 порядок равен 2.

– **Определяющая длина δ(H)** — расстояние между крайними фиксированными позициями: у схемы 1**0 она равна 3.

Пример

В задаче $f(x) = x^2$ схема 11*** объединяет все числа от 24 до 31, то есть самые приспособленные особи. Её порядок равен 2, определяющая длина — 1: это короткий шаблон малого порядка.

3 Теорема о схемах Холланда

Теперь сформулируем главный теоретический результат о генетическом алгоритме и прочитаем его по множителям.

$$m(H, t+1) \geq m(H, t) \cdot \frac{f(H)}{\bar{f}} \cdot \left[1 - p_c \frac{\delta(H)}{L-1} - o(H)\, p_m\right]$$

- Здесь m(H, t) — ожидаемое число особей схемы H в поколении t, f(H) — их средняя приспособленность, f̄ — средняя по популяции.
- Множитель f(H)/f̄ отражает отбор, а скобка — вероятность того, что скрещивание (p_c) и мутация (p_m) не разрушат схему.
- Утверждение: короткие схемы малого порядка с приспособленностью выше средней размножаются примерно экспоненциально.
- Пример: схема 11***, f(H)/f̄ ≈ 1,97, p_c = 0,8, p_m = 0,01, L = 5. Скобка равна 0,78, и одна копия даёт не меньше 1,54 копии.

Теорема даёт нижнюю оценку математического ожидания, а не точный прогноз: в конкретном запуске случайность может увести популяцию в сторону.

3 Что даёт теорема о схемах и чего она не даёт

Переведём формулу Холланда в практические выводы и честно назовём её ограничения.

– Отбор размножает схемы лучше средних, а операторы разрушают их тем сильнее, чем длиннее схема и чем больше в ней фиксированных позиций.

– Отсюда **гипотеза строительных блоков**: генетический алгоритм находит короткие удачные фрагменты хромосом и собирает из них решения.

– Практический вывод: связанные по смыслу параметры полезно ставить в хромосоме рядом, чтобы скрещивание их не разрывало.

– Теорема не гарантирует глобального оптимума и не говорит, за сколько поколений придёт сходимость; её критиковали и уточняли.

Пример

Скорость и дистанция торможения складского робота хороши только в паре. Если поставить их в хромосоме рядом, одноточечное скрещивание редко разорвёт эту пару, и удачная комбинация будет передаваться потомкам целиком.

? Как думаете?

Генетический алгоритм за несколько поколений вышел на плато: лучшая приспособленность перестала расти, хотя до оптимума далеко. Что стоит проверить первым?

- **A** Увеличить размер популяции вдвое
- **B** Повысить вероятность мутации и проверить разнообразие популяции
- **C** Запустить алгоритм на втрое большее число поколений
- **D** Сменить функцию приспособленности

? Как думаете?

Генетический алгоритм за несколько поколений вышел на плато: лучшая приспособленность перестала расти, хотя до оптимума далеко. Что стоит проверить первым?

- **A** Увеличить размер популяции вдвое
- **B** Повысить вероятность мутации и проверить разнообразие популяции
- **C** Запустить алгоритм на втрое большее число поколений
- **D** Сменить функцию приспособленности

ОТВЕТ

Разнообразие: плато обычно означает, что особи стали почти копиями. Помогут сильная мутация или более мягкий отбор, а не лишние поколения.

4 Эволюционные стратегии (ЭС)

Генетический алгоритм создавался для битовых строк; для непрерывных параметров в Германии в 1960-х придумали другой подход.

– **Эволюционные стратегии** (ЭС, Рехенберг и Швефель) работают с векторами вещественных чисел, а главный оператор у них — гауссова мутация.

– Простейшая (1+1)-ЭС держит одного родителя, добавляет к нему нормальный шум и оставляет потомка, если тот не хуже.

– В схеме (μ + λ) новое поколение выбирают из μ родителей и λ потомков вместе, поэтому лучшие решения не теряются.

– В схеме (μ, λ) при λ > μ родители отбрасываются; она лучше для шумных и меняющихся задач, где старая оценка могла быть завышена.

Ключевая идея

В хромосому ЭС записывают не только решение, но и параметры мутации σ. Они эволюционируют вместе с решением, и алгоритм сам подстраивает масштаб шага под рельеф целевой функции. Это называют самоадаптацией.

4

Правило одной пятой Рехенберга

Самый простой пример самонастройки: как выбирать силу мутации σ по статистике удачных шагов.

$$\sigma \leftarrow \begin{cases} \sigma / c, & \text{если доля успешных мутаций} > 1/5, \\ \sigma \cdot c, & \text{если доля успешных мутаций} < 1/5, \end{cases} \qquad 0 < c < 1$$

- Здесь σ — сила мутации (стандартное отклонение шума), c — постоянный множитель меньше единицы; успешная мутация улучшила решение.
- Если успешных мутаций больше пятой части, шаги можно делать смелее, и σ увеличивают; если меньше — шаги велики, и σ уменьшают.
- Пример: подбираем коэффициент регулятора, и из 20 последних проб удачны только 2, то есть 10 %, поэтому разброс проб сужают.
- Правило показывает идею самонастройки: алгоритм регулирует свой параметр по собственной статистике, без участия человека.

4 CMA-ES: адаптация ковариационной матрицы

Правило одной пятой меняет только длину шага. CMA-ES идёт дальше и подстраивает форму и направление облака мутаций.

– **CMA-ES** (Covariance Matrix Adaptation Evolution Strategy, Николаус Хансен) порождает мутации из многомерного нормального распределения.

– Ковариационная матрица этого распределения обновляется по истории успешных шагов, и облако мутаций вытягивается вдоль удачных направлений.

– Фактически метод оценивает локальную кривизну функции, как квазиньютоновские методы оценивают обратный гессиан, но без производных.

– Поэтому CMA-ES хорошо решает овражные и плохо обусловленные задачи и широко применяется для настройки контроллеров роботов.

Факт

Круглое облако мутаций в узком вытянутом овраге почти всегда промахивается мимо дна. CMA-ES растягивает облако вдоль оврага, и шаги идут по нему. Метод считают одним из сильнейших безградиентных методов для непрерывных задач.

5

Дифференциальная эволюция (DE)

Ещё один метод для непрерывных задач берёт масштаб шага прямо из популяции, а не из отдельного параметра σ.

$$v = x_{r_1} + F \cdot (x_{r_2} - x_{r_3})$$

- Здесь v — мутантный вектор, x_{r1}, x_{r2}, x_{r3} — три разных случайных вектора популяции, F — масштабирующий коэффициент примерно от 0,4 до 1,0.
- Затем v скрещивают с текущим вектором, и пробный вектор заменяет текущий, только если он не хуже (жадный отбор).
- Смысл: пока популяция разбросана, разности велики и шаги крупные; при сходимости шаги сами уменьшаются. Это самомасштабирование.
- Пример: два коэффициента регулятора; разность (0,9; 2,0) − (0,5; 1,0) при F = 0,5 даёт сдвиг (0,2; 0,5) к третьему вектору.

Дифференциальную эволюцию (Differential Evolution, DE) предложили Сторн и Прайс в 1990-х; у неё мало параметров, и её часто применяют для идентификации моделей роботов и настройки регуляторов.

5 Генетическое программирование (ГП)

Все методы выше подбирают числа. Генетическое программирование эволюционирует саму структуру решения — программу.

– **ГП** (генетическое программирование, Джон Коза, 1990-е) хранит особь как дерево: в узлах функции, в листьях константы и датчики.

– Скрещивание обменивает случайные поддеревья двух родителей, а мутация заменяет случайное поддерево новым.

– Приспособленность оценивает, насколько хорошо программа решает задачу, например насколько хорошо она управляет роботом.

– Типичная беда — **раздувание** (bloat): деревья растут без роста качества; с ним борются штрафом за сложность и лимитом глубины.

Пример

Для складского робота ГП может вырастить правило вида «если датчик спереди видит препятствие ближе 0,3 м, повернуть, иначе ехать к цели». Так получают законы управления и деревья поведения, когда их структура заранее неизвестна.

Конец первой части

Мы разобрали генетический алгоритм, теорему о схемах, эволюционные стратегии, CMA-ES, дифференциальную эволюцию и генетическое программирование. После перерыва — задачи с несколькими целями и эволюция поведения роботов.

Эволюционная робототехника

В первой части мы освоили эволюционные алгоритмы как способ оптимизации без производных; во второй применим их к компромиссам между целями и к выращиванию поведения роботов и роёв.

6 Многокритериальная оптимизация и доминирование по Парето

Инженерные задачи редко имеют одну цель; начнём вторую часть с того, как сравнивать решения по нескольким критериям.

– Обычно критериев несколько, и они конфликтуют: доставить заказ быстрее — значит потратить больше энергии и рисковать столкновением.

– Решение x **доминирует** решение y, если x не хуже y по всем критериям и строго лучше хотя бы по одному.

– **Множество Парето** — все решения, которые никто не доминирует; их образ в пространстве критериев называют **фронтом Парето**.

– Задача сводится к тому, чтобы найти весь фронт, а окончательный вариант выбирает человек по внешним соображениям.

Пример

Контроллер А доставляет заказ за 40 с и тратит 50 Дж, Б — за 60 с и 30 Дж. Ни один не доминирует другого: это две точки фронта Парето. Контроллер В с 70 с и 55 Дж хуже А по обоим критериям, и его отбрасывают.

6 NSGA-II: весь фронт Парето за один запуск

Эволюционный алгоритм держит целую популяцию, поэтому может приблизить весь фронт сразу; лучший известный пример — NSGA-II.

– **NSGA-II** (Non-dominated Sorting Genetic Algorithm II, К. Деб с соавторами) — генетический алгоритм для нескольких критериев.

– Первый механизм — **недоминируемая сортировка**: популяция делится на фронты по уровням доминирования, и первые фронты выигрывают отбор.

– Второй механизм — **оценка скученности** (crowding distance): внутри фронта выигрывают решения из редко заселённых участков.

– Алгоритм использует элитизм: перед отбором родителей и потомков объединяют, поэтому найденные компромиссы не теряются.

Пример

Для маршрута складского робота NSGA-II за один запуск выдаёт набор вариантов от «короткий, но рядом с людьми» до «длиннее, но безопасный». Инженер видит цену каждого компромисса и выбирает осознанно.

7 Сравнение эволюционных методов

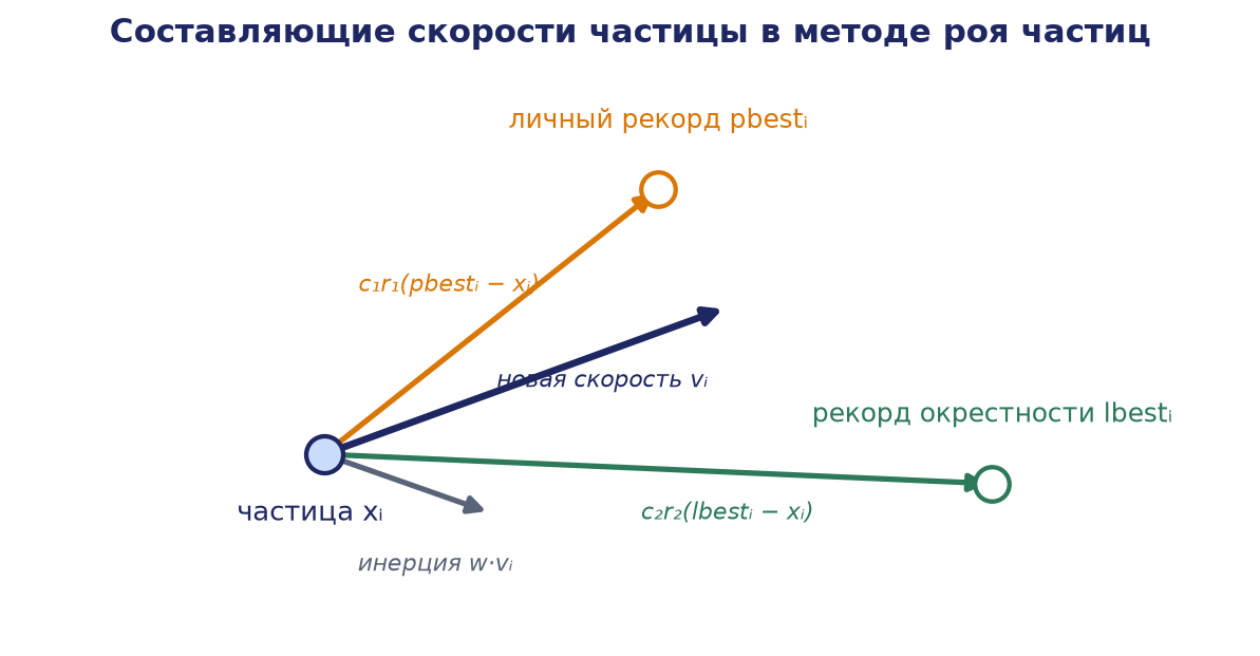
Мы познакомились с шестью семействами методов; сведём их в таблицу, чтобы было видно, что брать для какой задачи.

Метод	Пространство и представление	Главный оператор	Где применять
ГА (генетический алгоритм)	Дискретное или смешанное; строка бит или чисел	Скрещивание и мутация	Маршруты, комбинаторные задачи
ЭС (эволюционная стратегия)	Непрерывное; вектор чисел	Гауссова мутация с самоадаптацией σ	Настройка непрерывных параметров
CMA-ES	Непрерывное; вектор и ковариационная матрица	Адаптация ковариации	Овражные, плохо обусловленные задачи
DE (дифференциальная эволюция)	Непрерывное; вектор чисел	Разностная мутация	Инженерная оптимизация, идентификация
ГП (генетическое программирование)	Пространство программ; дерево	Обмен поддеревьями	Синтез контроллеров и правил
NSGA-II	Несколько критериев; вектор и ранг Парето	Сортировка и скученность	Многокритериальные компромиссы

Теорема об отсутствии бесплатных завтраков (No Free Lunch) утверждает: если усреднить по всем возможным задачам, любые два алгоритма поиска работают одинаково. Поэтому универсально лучшего метода нет, и его подбирают под задачу.

7 Рой частиц: улучшение без отбора

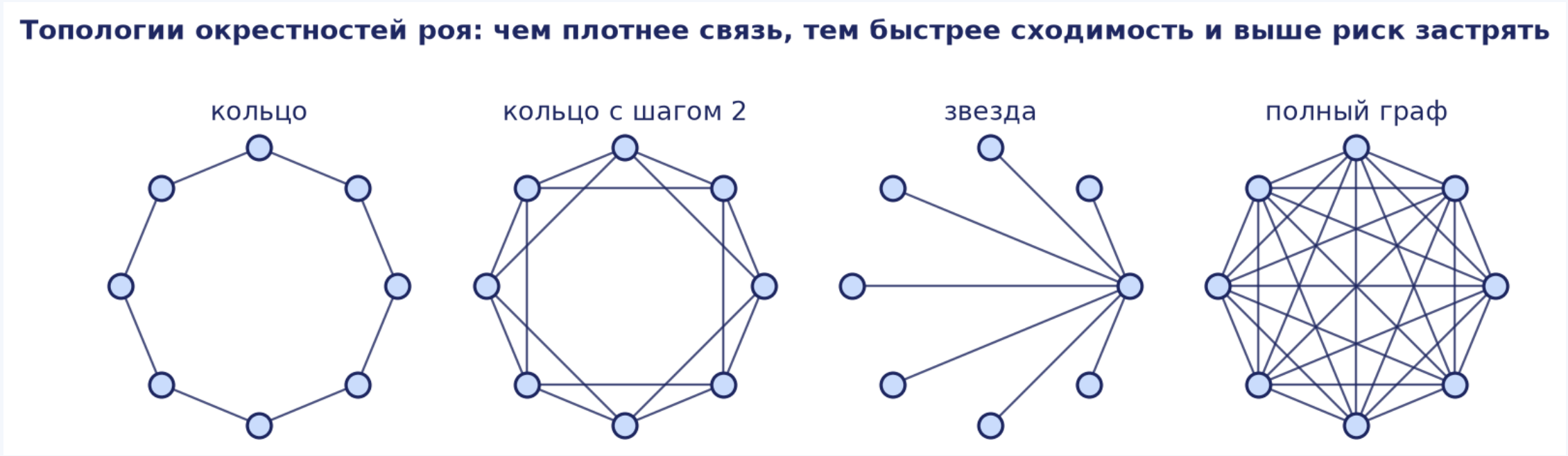
На пятой лекции мы разбирали метод роя частиц (PSO); сравним его с эволюцией, ведь задачи он решает те же.



Метод роя частиц (Particle Swarm Optimization, PSO) никого не отбраковывает: каждая частица-решение сдвигается суммой трёх составляющих — инерции своего движения, притяжения к собственному лучшему результату и притяжения к лучшему результату соседей. Генетический алгоритм, наоборот, размножает лучших и удаляет худших. Философии разные, а классы задач одни и те же.

7 Топология соседства влияет на поиск

Кого частица считает соседом, так же важно, как граф связи для консенсуса на четвёртой лекции.



Географическое соседство определяется расстоянием между частицами в пространстве поиска, социальное — заранее заданной структурой связей, независимо от положения. Плотная топология быстро распространяет лучший результат и ускоряет сходимость, но повышает риск, что весь рой застрянет в локальном оптимуме.

8 Идея эволюционной робототехники

Главный поворот лекции: применим эволюцию не к отдельным числам, а к поведению робота целиком.

– **Эволюционная робототехника** (Нольфи и Флореано) автоматически синтезирует систему управления робота, а иногда и его форму.

– Инженер задаёт функцию приспособленности на уровне цели: «пройти как можно дальше», «собрать больше объектов», «не сталкиваться».

– Контроллер обычно представлен нейронной сетью, и эволюционирует хромосома, в которой записаны веса её связей.

– Эволюция нередко находит неочевидные и экономные решения, использующие взаимодействие тела робота, контроллера и среды.

Ключевая идея

Проектировщик перестаёт описывать, **как** робот должен действовать, и описывает только, **что** считать хорошим результатом. Вместо программы объезда стеллажей мы награждаем за доставку без столкновений, а способ находит алгоритм.

8 Цикл оптимизации контроллера

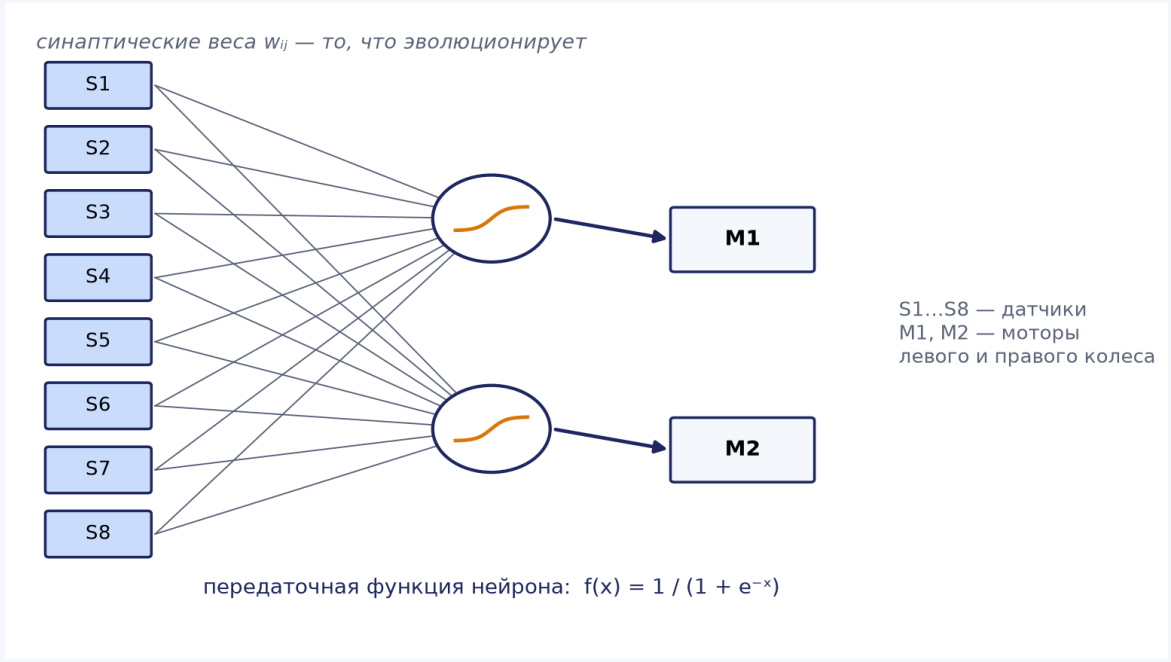
Технически оптимизатор и испытательный стенд работают в замкнутом цикле; посмотрим, где в нём узкое место.



Метаэвристика (генетический алгоритм или метод роя частиц) порождает набор кандидатов; каждый разворачивается в контроллер, испытывается в симуляции или на реальном роботе и получает оценку приспособленности, после чего набор обновляется. Узкое место — испытание: операторы алгоритма работают доли секунды, а одно испытание длится секунды или десятки секунд.

8 Нейросетевой контроллер робота

Что именно эволюционирует внутри кандидата? Чаще всего — веса небольшой нейронной сети.



Сигналы восьми датчиков через синаптические веса w_{ij} поступают на нейроны с сигмоидальной функцией $f(x) = 1/(1 + e^{-x})$, выходы которых задают команды двум моторам. Хромосома — это набор весов. Нейросеть служит здесь не для распознавания, а как способ записать поведение числами.

8 Нейроэволюция: эволюция весов и структуры

Если размер сети заранее неочевиден, можно эволюционировать и её структуру; это направление называют нейроэволюцией.

– **Нейроэволюция** создаёт нейронные сети эволюцией и настраивает не только веса связей, но и топологию: число нейронов и связей.

– Простое скрещивание сетей разной структуры разрушительно: одна и та же функция может быть записана разными наборами генов.

– Новая связь или нейрон поначалу почти всегда ухудшают приспособленность, и без защиты отбор сразу удаляет такую новинку.

– Самый известный метод, решающий эти проблемы, — **NEAT** (NeuroEvolution of Augmenting Topologies, Стэнли и Миикkулайнен).

Пример

Для робота, объезжающего препятствия, NEAT может начать с сети без скрытых нейронов, где датчики напрямую связаны с моторами, и добавлять нейроны, только если это повышает приспособленность.

8 Три идеи алгоритма NEAT

NEAT (нейроэволюция наращиваемых топологий) отвечает на каждую трудность нейроэволюции отдельным механизмом.

Трудность	Решение в NEAT	Что это даёт
Одну сеть можно закодировать по-разному, и скрещивание её ломает	Исторические маркеры: каждый ген связи помечен моментом появления	Хромосомы разной структуры корректно выравниваются
Новая структура сначала работает хуже и сразу вытесняется	Видообразование: конкуренция идёт в основном внутри вида	Новинка получает время «созреть»
Большая сеть сразу даёт огромное пространство поиска	Усложнение от простого: старт с сетей без скрытых нейронов	Сложность растёт, только когда окупается качеством

Первую трудность называют проблемой конкурирующих соглашений (competing conventions), а маркеры — номерами инноваций (innovation numbers).

9 Проблема задания целевой функции

Мы сказали, что инженер задаёт только цель. Оказывается, правильно задать цель — самая трудная часть работы.

– Если функция слишком подробная, вы по сути вручную запрограммировали решение, и эволюция ничего нового не найдёт.

– Если функция слишком общая, например «1 балл, если доставлены все заказы», в начале все особи получают ноль, и отбору не из чего выбирать.

– Алгоритм оптимизирует ровно то, что вы измеряете, и всегда находит **лазейку**, если она в функции есть.

– Поэтому настройка функции приспособленности часто занимает больше времени, чем сама оптимизация.

Кейс

Робот, которого награждают за пройденное по одометрии расстояние, может научиться вращаться на месте: колёса крутятся, счётчик пути растёт, формально задача выполнена. Награждать нужно за смещение к цели, а не за обороты колёс.

9 Проблема назначения вклада

Для группы роботов к трудности с целью добавляется ещё одна: кому из роботов приписать общий успех.

– Задачу решает группа, а приспособленность часто одна на всех, поэтому неясно, чей вклад был полезным, а чей вредным.

– При **командном отборе** все получают одну оценку: кооперация поощряется, но простоявший смену робот оценён так же, как работавший.

– При **индивидуальном отборе** каждого оценивают по своему вкладу: отбор идёт быстрее, но альтруистичная кооперация возникает труднее.

– На седьмой лекции та же трудность называлась распределением заслуг (credit assignment) в обучении с подкреплением.

Пример

Два складских робота вместе подняли тяжёлый контейнер. Если оценивать только того, кто довёз контейнер до ворот, помогавший робот выглядит бесполезным, и его удачное поведение будет отброшено.

9 Почему вклад трудно оценить

Есть и более тонкая причина: робот часто просто не видит, что изменило среду — он сам или его сосед.



При ограниченной связи или частичной наблюдаемости робот не может отделить изменения среды, вызванные его собственными действиями, от изменений, вызванных соседями. Например, проход освободился, потому что соседний робот увёз паллету, а наш робот приписал успех своему манёвру. Поэтому награды и штрафы расходятся с реальным вкладом.

9 Однородные и разнородные группы

Ещё одно решение принимают до начала эволюции: давать всем роботам один контроллер или каждому свой.

Однородная группа (общий контроллер)

– Все роботы несут один и тот же контроллер, который эволюционирует как целое.

– Пространство поиска не растёт с числом роботов, поэтому сходимость быстрее.

– Специализация и разделение труда невозможны: все роботы ведут себя одинаково.

Разнородная группа (свои контроллеры)

– У каждого робота или у каждой касты роботов свой набор параметров.

– Пространство поиска растёт с числом роботов или каст, и сходимость медленнее.

– Возможна специализация: одни роботы возят коробки, другие расчищают проходы.

– Как мы видели на седьмой лекции, специализация особенно выгодна в небольших группах.

9 Дорогая оптимизация и шум в оценках

Даже с правильной целью и правильной группой остаётся практическая беда: испытания дороги и неточны.

– Одно испытание контроллера длится секунды или десятки секунд, а алгоритму нужны сотни и тысячи испытаний.

– Оценка зашумлена: тот же контроллер в двух прогонах доставляет разное число коробок из-за расстановки грузов и шума датчиков.

– Шум сбивает отбор: посредственный контроллер, которому повезло в одном прогоне, вытесняет действительно хороший.

– Помогают повторные оценки лидеров, турнирный отбор и схема (μ, λ), которая «забывает» завышенные оценки, но повторы удорожают поиск.

? Инженерная ситуация

Контроллер складского робота, выращенный эволюцией в симуляторе, отлично работает в симуляции и полностью проваливается на реальном роботе. Какая причина наиболее вероятна?

A Слишком мало поколений: нужно учить дольше

B Эволюция подстроилась под неточности симулятора

C Выбран неудачный метод отбора

D Слишком большая популяция

Инженерная ситуация

Контроллер складского робота, выращенный эволюцией в симуляторе, отлично работает в симуляции и полностью проваливается на реальном роботе. Какая причина наиболее вероятна?

A Слишком мало поколений: нужно учить дольше

B Эволюция подстроилась под неточности симулятора

C Выбран неудачный метод отбора

D Слишком большая популяция

ОТВЕТ

Эволюция нашла и использовала неточности модели, которых в реальности нет. Это разрыв между симуляцией и реальностью; долгое обучение его только углубит.

10 Разрыв между симуляцией и реальностью

Квиз подвёл нас к главной практической проблеме эволюционной робототехники; разберём её причины и способы борьбы.

– **Разрыв реальности** (reality gap) — расхождение поведения контроллера в симуляции и на роботе из-за неточных моделей физики и датчиков.

– Эволюция находит и **использует** эти неточности, например нереально малое трение или датчики без шума.

– Шум в симуляции и случайный разброс параметров среды заставляют решение работать в целом диапазоне условий.

– Заключительное дообучение на реальном роботе исправляет то, что симуляция всё равно не учла.

Ключевая идея

Чем точнее контроллер подогнан под один симулятор, тем хуже он переносится. Поэтому симуляцию полезно намеренно «портить»: менять от прогона к прогону трение пола, массу груза и шум дальномеров.

10 Модульный автоматический дизайн

Другой способ сократить разрыв — не давать алгоритму лишней свободы, а собирать поведение из проверенных блоков.

– Контроллер собирается из заранее проверенных поведенческих блоков, например «ехать к цели», «объехать препятствие», «остановиться».

– Алгоритм оптимизирует не веса сети, а то, какие блоки взять, как соединить их переходами и с какими параметрами запустить.

– Пространство поиска резко сужается, и найденный контроллер легче понять и проверить.

– Переносимость на реальных роботов заметно выше, потому что каждый блок по отдельности прост и предсказуем.

Ключевая идея

Модульность играет роль регуляризации, как ограничение класса моделей в машинном обучении: в маленьком пространстве проверенных блоков алгоритму негде найти лазейку в симуляторе.

10 Сравнение подходов к проектированию поведения

Сведём вместе ручное программирование, модульный дизайн, нейроэволюцию и обучение с подкреплением.

Подход	Кто задаёт способ действия	Переносимость на робота	Прозрачность
Ручное программирование	Инженер полностью	Высокая	Полная
Модульный автоматический дизайн	Инженер задаёт блоки, алгоритм — их сборку	Хорошая	Хорошая
Нейроэволюция	Алгоритм полностью	Слабая	Низкая
Обучение с подкреплением	Алгоритм полностью	Средняя	Низкая

Чем больше свободы у алгоритма, тем интереснее найденные решения и тем труднее их перенести на реального робота и объяснить.

11 Оффлайн- и онлайн-эволюция

До сих пор эволюция шла заранее, на компьютере. Но среда на складе меняется, значит, адаптация нужна и во время работы.

– **Оффлайн-эволюция** идёт заранее, обычно в симуляции на внешнем компьютере; лучший контроллер затем загружают в роботов.

– Она проста и допускает большие популяции, но не даёт адаптации во время работы и страдает от разрыва реальности.

– **Онлайн-эволюция** идёт во время эксплуатации, прямо на роботах, и подстраивает поведение под новые препятствия, износ приводов и отказы.

– Её трудности: слабые бортовые процессоры, требование быстрого отклика, забывание старого опыта и безопасность проб на настоящем роботе.

Пример

Эволюция может вырастить начальный контроллер складского робота оффлайн, а онлайн-обучение с подкреплением — дообучить его на конкретном складе. Такую гибридизацию считают перспективным направлением.

11 Воплощённая эволюция (embodied evolution)

Самая «роевая» форма онлайн-эволюции: популяция физически совпадает с роем, а операторами служат сами роботы.

– В **воплощённой эволюции** (Уотсон, Фикичи и Полак) нет внешнего компьютера: каждый робот несёт свой геном, то есть свой контроллер.

– При встрече роботы обмениваются частями геномов по локальной беспроводной связи, и успешный робот передаёт геном с большей вероятностью.

– Мутация применяется локально к полученным геномам, поэтому отбор, скрещивание и мутация идут распределённо и асинхронно.

– Схема масштабируема и устойчива к отказам роботов — это естественная форма эволюции для распределённых робототехнических систем (PPTC).

Пример

Мерой успеха складского робота может служить накопленная «энергия», например число доставленных коробок. Встретившись у зарядной станции, успешный робот чаще передаёт соседу свой контроллер, и удачное поведение расходится по рою.

11 Режимы эволюции в роях роботов

Сравним три режима эволюции по месту, времени и цене, чтобы понимать, какой выбрать для конкретной задачи.

Критерий	Оффлайн-эволюция	Онлайн-эволюция	Воплощённая эволюция
Где идёт процесс	На внешнем компьютере	На роботах	Распределённо по рою
Когда идёт процесс	До эксплуатации	Во время работы	Во время работы
Управление	Централизованное	Локальное или централизованное	Полностью децентрализованное
Адаптация к изменениям	Отсутствует	Есть	Есть
Разрыв реальности	Значителен	Мал	Отсутствует
Масштабируемость	Ограничена	Средняя	Высокая
Стоимость проб	Низкая (симуляция)	Высокая (реальные пробы)	Высокая (реальные пробы)

“

Эволюция изобретательна ровно настолько, насколько точна ваша функция приспособленности: если в ней есть лазейка, алгоритм её найдёт.

Главный практический вывод лекции

11

Где эволюция уместна, а где нет

В завершение честно определим место эволюционных методов среди других инструментов инженера.

– Эволюция уместна, когда нет модели и производной, а у целевой функции много локальных экстремумов, разрывы или шум.

– Она уместна, когда структура решения заранее неизвестна и нужно найти неочевидное поведение, например правила для роя.

– Она неуместна, когда задача решается градиентным методом: тот потребует на порядки меньше вычислений целевой функции.

– Она неуместна, когда нужны гарантии качества и сертификация: найденный контроллер трудно объяснить и доказать его безопасность.

Пример

Подобрать два коэффициента регулятора по гладкой модели проще градиентным методом. Вырастить правила, по которым двадцать складских роботов делят проходы без центрального диспетчера, — задача как раз для эволюции.

Выводы по лекции

1. Эволюционные алгоритмы ищут решение отбором и случайными изменениями там, где нет модели и производной.
2. Генетический алгоритм состоит из кодирования, отбора, скрещивания и мутации; сильнее всего влияет кодирование.
3. По теореме о схемах короткие удачные шаблоны хромосом размножаются в популяции почти экспоненциально.
4. Эволюционные стратегии, CMA-ES и дифференциальная эволюция сами подстраивают шаг поиска к рельефу функции.
5. В многокритериальных задачах ответ — фронт Парето, и NSGA-II строит его за один запуск.
6. В эволюционной робототехнике инженер задаёт цель, а платит за это прозрачностью и риском лазеек.
7. Главные трудности — назначение вклада, шум и разрыв реальности; онлайн-эволюция даёт адаптацию на месте.

Вопросы для самопроверки

1. В каких случаях эволюционный алгоритм предпочтительнее градиентного метода?

2. Чем генотип отличается от фенотипа? Приведите пример для складского робота.

3. Почему рулеточный отбор чувствителен к масштабу приспособленности, а турнирный — нет?

4. Зачем нужен элитизм и что происходит без него?

5. Сформулируйте теорему о схемах. Как в неё входят порядок и определяющая длина схемы?

6. Чем схема $(\mu + \lambda)$ отличается от (μ, λ) и когда лучше каждая из них?

7. Почему говорят о самомасштабировании шага в дифференциальной эволюции?

8. Что такое фронт Парето и на каких двух механизмах основан NSGA-II?

9. Какие трудности нейроэволюции решает NEAT и какими средствами?

10. Что такое разрыв реальности и как его уменьшают?